

Numerical Analysis

Theoretical Foundations

Root-Finding · Systems of Equations · Interpolation

TEACHER

Dr. Hisam Uddin Shaikh

Chairman, Dept. of Mathematics

COMPOSED BY

Nadir Ali Khan

BS-IV Mathematics

DESIGN

Palwasha Ashique

BS-IV Mathematics

Topics Covered:

Chapter 1 — Root-Finding Methods

1. Bisection Method
2. Regula Falsi Method
3. Newton-Raphson Method
4. Secant Method
5. Fixed Point Iteration

System of Linear Equations

1. Gaussian Elimination
2. LU Decomposition
3. Jacobi Iterative Method
4. Gauss-Seidel Method

Chapter 2 — Interpolation

1. Finite Differences
2. Newton's Forward Formula
3. Newton's Backward Formula
4. Gauss Forward Formula
5. Gauss Backward Formula
6. Stirling's Formula
7. Bessel's Formula
8. Lagrange's Formula
9. Newton's Divided Difference

Each topic covers: historical background, mathematical foundation, full derivation, convergence proofs, error analysis, variants, failure cases, and practical considerations.

Scope of this Document: This document provides a rigorous, research-level treatment of numerical root-finding methods, systems of linear equations, and interpolation formulas. Each method is examined from multiple perspectives: its historical origins, complete mathematical derivation, formal convergence proofs, error analysis, worked examples, and practical implementation guidelines. The treatment is intended for advanced undergraduate students of mathematics (BS-IV level).

Table of Contents

Chapter 1 — Root-Finding Methods

Introduction to Numerical Root Finding	5
Bisection Method	8
Regula Falsi Method.....	12
Newton-Raphson Method.....	16
Secant Method	20
Fixed Point Iteration	24
Comprehensive Comparison of All Methods	29

System of Linear Equations

Introduction to Systems.....	33
Gaussian Elimination	34
LU Decomposition	36
Jacobi Iterative Method	38
Gauss-Seidel Method	40

Chapter 2 — Interpolation

Introduction to Interpolation.....	43
Finite Differences	43
Newton's Forward Interpolation Formula.....	45
Newton's Backward Interpolation Formula.....	48
Central Differences — Gauss Forward Formula.....	50
Central Differences — Gauss Backward Formula.....	53
Comprehensive Comparison of Interpolation Formulas	55
Stirling's Interpolation Formula.....	56
Stirling's Interpolation Formula.....	56
Bessel's Interpolation Formula	59

Bessel's Interpolation Formula	59
Lagrange's Interpolation Formula.....	61
Lagrange's Interpolation Formula.....	61
Newton's Divided Difference Method	63
Newton's Divided Difference Method	63
Comprehensive Comparison of All Interpolation Formulas.....	65

Introduction to Numerical Root Finding

Definition — Root of a Function

A **root** (or **zero**) of a function f is a value x^* such that

$$f(x^*) = 0$$

Equivalently, x^* is the x -coordinate of the intersection of $y = f(x)$ with the x -axis.

Why Numerical Methods?

Many real-world equations cannot be solved in closed form. For example:

$$x^3 - 2x - 5 = 0, \quad e^x - 3x = 0, \quad x \sin x - 1 = 0$$

Numerical methods produce *approximate* solutions to any desired precision through an iterative process.

Theorem — Intermediate Value Theorem (IVT)

Let $f : [a, b] \rightarrow \mathbb{R}$ be continuous. If $f(a) \cdot f(b) < 0$, then there exists at least one point $x^* \in (a, b)$ such that $f(x^*) = 0$.

Proof sketch: Define $S = \{x \in [a, b] : f(x) < 0\}$. Since $f(a) < 0$ (WLOG), $a \in S$ so $S \neq \emptyset$. Also S is bounded above by b . Let $x^* = \sup S$. By continuity of f , one can show $f(x^*) = 0$: if $f(x^*) > 0$, there is a neighbourhood where $f > 0$, contradicting x^* being the supremum; if $f(x^*) < 0$, there is a neighbourhood where $f < 0$, so points larger than x^* are in S , again a contradiction.

Important: The IVT guarantees *existence* but not *uniqueness*. Also, the sign-change condition is *sufficient* but not necessary — a root may exist without a sign change (e.g. $f(x) = (x - 1)^2 \geq 0$ everywhere).

Types of Errors in Numerical Computation

Definition — Absolute Error

$|e_n| = |x_n - x^*|$, where x_n is the computed approximation and x^* is the true root.

Definition — Relative Error

$\frac{|e_n|}{|x^*|} = \frac{|x_n - x^*|}{|x^*|}$ ($x^* \neq 0$). Dimensionless; more meaningful when x^* is very large or very small.

Definition — Truncation vs. Round-off Error

Truncation error arises from approximating an infinite process (e.g. truncating a Taylor series). It is inherent to the method.

Round-off error arises from finite floating-point representation (≈ 15 – 16 significant digits in 64-bit double precision). Subtraction of nearly equal numbers causes **catastrophic cancellation**, drastically reducing accuracy.

Condition Number of a Root**Definition — Condition Number**

For a root x^* with $f'(x^*) \neq 0$, the (absolute) condition number is $\kappa = 1/|f'(x^*)|$. If the function is perturbed by δ , the root shifts by approximately $\delta/|f'(x^*)|$.

- **Well-conditioned** ($|f'(x^*)|$ large): f crosses the axis steeply — small perturbations shift the root very little.
- **Ill-conditioned** ($f'(x^*) \approx 0$): f is nearly flat near the root — small perturbations can shift the root enormously. Multiple roots ($f'(x^*) = 0$) are the extreme case.

Stopping Criteria

Five common criteria used in practice:

1. **Interval width:** $|b_n - a_n| < \varepsilon_1$ — guarantees root lies in a small interval (bracketing methods only).
2. **Function value:** $|f(x_n)| < \varepsilon_2$ — function is close to zero (can fail if f is flat).
3. **Absolute change:** $|x_n - x_{n-1}| < \varepsilon_3$ — consecutive iterates close (can fail for slow convergence).
4. **Relative change:** $|x_n - x_{n-1}|/|x_n| < \varepsilon_4$ — scale-invariant version.
5. **Maximum iterations:** $n \geq N_{\max}$ — safety net to prevent infinite loops.

In practice, a combination of criteria is used. Stop when *any* (or *all*) are satisfied.

Classification of Root-Finding Methods

Category	Methods	Key Characteristic
Bracketing Methods	Bisection, Regula Falsi	Require $f(a) \cdot f(b) < 0$; guaranteed convergence
Open Methods	Newton-Raphson, Secant	Start from 1–2 initial guesses; may diverge
Fixed Point Methods	Fixed Point Iteration	Rewrite $f(x) = 0$ as $x = g(x)$; converge if $ g' < 1$
Hybrid Methods	Brent's Method, Illinois	Combine bracketing safety with open-method speed
Higher-Order	Halley's, Steffensen	Cubic or better convergence; specialised use

Convergence Terminology

Definition — Order of Convergence

A sequence $\{x_n\}$ converging to x^* has **order of convergence** $p \geq 1$ if there exists a constant $C > 0$ (the **asymptotic error constant**) such that

$$\lim_{n \rightarrow \infty} \frac{|e_{n+1}|}{|e_n|^p} = C, \quad e_n = x_n - x^*$$

- $p = 1, C < 1$: **Linear convergence** — error reduced by factor C each step.
- $p = 1, C = 1$: **Sublinear** — very slow, may barely move.
- $1 < p < 2$: **Superlinear** — between linear and quadratic.
- $p = 2$: **Quadratic** — correct digits double each step.
- $p = 3$: **Cubic** — correct digits triple each step.

Practical Illustration: Starting with $|e_0| = 0.1$ and asymptotic constant $C \approx 1$:

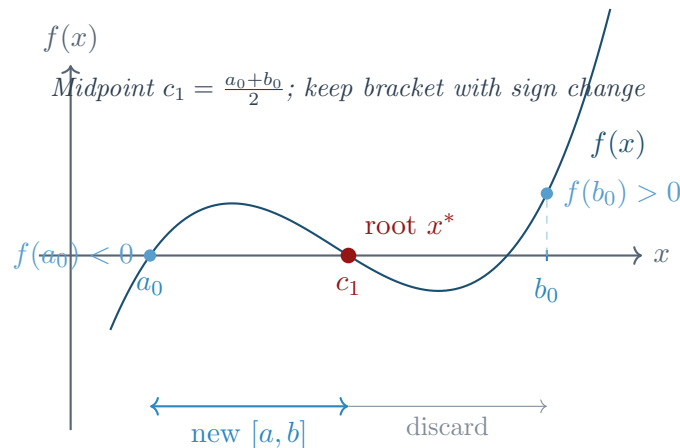
n	Linear ($p = 1, C = 0.5$)	Superlinear ($p \approx 1.618$)	Quadratic ($p = 2$)
0	10^{-1}	10^{-1}	10^{-1}
1	5×10^{-2}	$\sim 10^{-1.6}$	10^{-2}
2	2.5×10^{-2}	$\sim 10^{-2.6}$	10^{-4}
3	1.25×10^{-2}	$\sim 10^{-4.2}$	10^{-8}
4	6.25×10^{-3}	$\sim 10^{-6.9}$	10^{-16}

METHOD 01

Bisection Method

Historical Background

The Bisection Method embodies one of humanity's oldest problem-solving strategies: *divide and conquer*. Its mathematical foundation — the Intermediate Value Theorem — was first rigorously proved by **Bernard Bolzano** (1817, *Rein analytischer Beweis*) and independently refined by **Augustin-Louis Cauchy** (*Cours d'analyse*, 1821), motivated by the need to put calculus on rigorous foundations. The algorithm is a direct computational translation of Bolzano's proof. In computer science, the same idea appears as *binary search* — invented independently for sorted arrays. The method has been implemented in virtually every scientific computing environment and remains the gold standard for safe, guaranteed root finding.



Mathematical Foundation

Definition — Bisection Algorithm

Given a continuous function f on $[a_0, b_0]$ with $f(a_0) \cdot f(b_0) < 0$, at each step $n = 0, 1, 2, \dots$:

1. Compute the midpoint: $c_n = \frac{a_n + b_n}{2}$
2. If $f(c_n) = 0$: exact root found; stop.
3. If $f(a_n) \cdot f(c_n) < 0$: root in $[a_n, c_n]$; set $a_{n+1} = a_n$, $b_{n+1} = c_n$.
4. If $f(c_n) \cdot f(b_n) < 0$: root in $[c_n, b_n]$; set $a_{n+1} = c_n$, $b_{n+1} = b_n$.
5. Repeat until stopping criterion is met.

The sequence of midpoints $\{c_n\}$ converges to a root x^* .

Theorem — Convergence of Bisection — Nested Interval Theorem

Let f be continuous on $[a_0, b_0]$ with $f(a_0) \cdot f(b_0) < 0$. The Bisection Method produces nested intervals $[a_n, b_n]$ satisfying:

1. **Nesting:** $[a_0, b_0] \supset [a_1, b_1] \supset [a_2, b_2] \supset \dots$
2. **Bracketing:** $f(a_n) \cdot f(b_n) < 0$ for all n .
3. **Width:** $b_n - a_n = \frac{b_0 - a_0}{2^n}$
4. **Convergence:** $\{a_n\}$ (non-decreasing, bounded above) and $\{b_n\}$ (non-increasing, bounded below) both converge to the same limit $x^* \in [a_0, b_0]$ with $f(x^*) = 0$.

Proof of (iv): By (iii), $b_n - a_n \rightarrow 0$. By the Monotone Convergence Theorem, $\{a_n\} \rightarrow x^*$ and $\{b_n\} \rightarrow x^*$. By (ii) and continuity of f : taking limits gives $[f(x^*)]^2 \leq 0$, so $f(x^*) = 0$.

□

Error Analysis

After n iterations starting from interval $[a_0, b_0]$, the interval has length:

$$b_n - a_n = \frac{b_0 - a_0}{2^n}$$

The root x^* always lies inside $[a_n, b_n]$, so the **absolute error bound** is:

$$|x_n - x^*| \leq \frac{b_0 - a_0}{2^n}$$

Theorem — Number of Iterations Required

To achieve an error less than ε , the number of iterations n must satisfy:

$$\frac{b_0 - a_0}{2^n} < \varepsilon \quad \implies \quad n > \frac{\ln\left(\frac{b_0 - a_0}{\varepsilon}\right)}{\ln 2} = \log_2\left(\frac{b_0 - a_0}{\varepsilon}\right)$$

Convergence Analysis

Theorem — Linear Convergence of Bisection

Bisection converges **linearly** with convergence ratio $\frac{1}{2}$:

$$|e_{n+1}| \leq \frac{1}{2}|e_n| \quad \text{where} \quad e_n = x_n - x^*$$

Each iteration reduces the error by exactly half, regardless of the behavior of f .

Key Insight: Bisection ignores the actual values $f(a)$ and $f(b)$ — it only uses their *signs*. This makes it very robust (guaranteed convergence) but also inefficient (slow, wastes information about how large or small f is).

Stopping Criteria Specific to Bisection

For Bisection, the interval-width criterion is always preferred (we have a guaranteed bound):

1. **Primary:** $|b_n - a_n| < 2\varepsilon$, ensuring $|c_n - x^*| < \varepsilon$.
2. **Secondary:** Also check $|f(c_n)| < \varepsilon_f$ for a function tolerance.
3. **Safety:** Always set a maximum iteration count $N_{\max}$ (computable from the formula above).

Caution: The relative change criterion $|c_n - c_{n-1}|/|c_n|$ can be unreliable when $c_n \approx 0$.

When Bisection Fails

1. **Discontinuous functions:** IVT requires continuity. $f(x) = 1/x$ has $f(-1) = -1$ and $f(1) = 1$, but f has a pole at 0, not a root.
2. **Even-multiplicity roots:** $f(x) = (x - x^*)^2 \geq 0$ — no sign change, Bisection cannot locate x^* .
3. **Multiple roots in bracket:** If there are an odd number of roots in $[a, b]$, Bisection finds one, but not necessarily the desired one.
4. **Flat functions:** Round-off in evaluating $\text{sign}(f(c_n))$ may be incorrect if $f(c_n) \approx 0$ at machine precision.

Application: Eigenvalue Computation via Sturm Sequences

A remarkable application of Bisection is computing eigenvalues of symmetric tridiagonal matrices. The **Sturm sequence property** allows the number of eigenvalues less than any value λ to be computed in $\mathcal{O}(n)$ operations. Combined with Bisection on the parameter λ , this gives a numerically stable $\mathcal{O}(n \log(1/\varepsilon))$ algorithm — still used in modern linear algebra libraries (LAPACK's `dstebe`).

Advantages

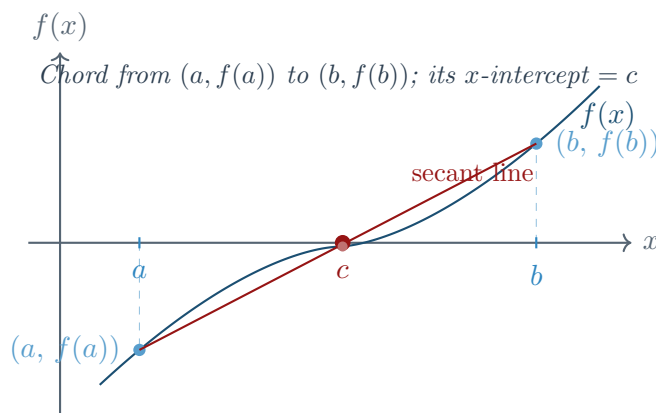
- Guaranteed convergence (IVT)
- Simple to implement correctly
- No derivative required
- A priori error bound (predictable)
- Never diverges or gets stuck
- Number of iterations known in advance
- Works for any continuous f

Disadvantages

- Slow: linear convergence, factor = 0.5
- Requires initial bracket with sign change
- Ignores function magnitude (wastes info)
- Cannot find complex roots
- Cannot detect even-multiplicity roots
- Only finds one root per bracket
- Not easily extended to higher dimensions

Regula Falsi Method (False Position)

The Regula Falsi (Latin for *Rule of False Position*) is one of the **oldest known root-finding algorithms**, appearing in ancient Egyptian mathematics (circa 1550 BCE, Rhind Papyrus), ancient Babylonian texts, and Indian mathematics (Bakhshali manuscript). Medieval Arab mathematicians including Al-Khawarizmi developed it further. It predates calculus by thousands of years and was the dominant algebraic technique until Newton's method emerged in the 17th century. It was formalized in its modern bracket-preserving form during the development of numerical analysis.



Mathematical Foundation

Definition — Geometric Interpretation

Rather than taking the midpoint of $[a, b]$, Regula Falsi draws a **straight line** (secant line) connecting the points $(a, f(a))$ and $(b, f(b))$, then uses the x -intercept of this line as the next approximation.

From the equation of the line through two points:

$$\frac{x - a}{b - a} = \frac{0 - f(a)}{f(b) - f(a)}$$

Solving for x gives the **false position formula**:

$$x_n = \frac{a f(b) - b f(a)}{f(b) - f(a)} = b - f(b) \cdot \frac{b - a}{f(b) - f(a)}$$

Derivation — Formula from Similar Triangles

Consider the two right triangles formed by the secant line cutting the x -axis at x_n :

$$\frac{x_n - a}{b - x_n} = \frac{|f(a)|}{|f(b)|}$$

$$(x_n - a)|f(b)| = (b - x_n)|f(a)|$$

$$x_n[|f(b)| + |f(a)|] = b|f(a)| + a|f(b)|$$

When $f(a) < 0$ and $f(b) > 0$, accounting for signs gives the standard formula.

The Algorithm

Definition — Regula Falsi Algorithm

Given $[a_0, b_0]$ with $f(a_0) \cdot f(b_0) < 0$:

1. Compute: $c_n = \frac{a_n f(b_n) - b_n f(a_n)}{f(b_n) - f(a_n)}$
2. If $f(c_n) = 0$: root found; stop.
3. If $f(a_n) \cdot f(c_n) < 0$: set $b_{n+1} = c_n$, $a_{n+1} = a_n$.
4. Else: set $a_{n+1} = c_n$, $b_{n+1} = b_n$.
5. Repeat until $|f(c_n)| < \varepsilon$ or $|b_n - a_n| < \delta$.

Convergence Analysis and Stagnation

Regula Falsi always converges (bracket maintained). However, it suffers from a well-known pathology:

Theorem — One-Sided Convergence — Stagnation Theorem

If f is convex ($f'' > 0$) on $[a, b]$ with root $x^* \in (a, b)$ and $f(a) < 0 < f(b)$, then b is retained at every step: $b_n = b_0$ for all n . Only the left endpoint $a_n \rightarrow x^*$. Convergence is linear with rate:

$$\lim_{n \rightarrow \infty} \frac{|e_{n+1}|}{|e_n|} = 1 - \frac{f'(x^*)}{f'(x^*) + K} < 1$$

where K is related to the curvature of f at x^* . This ratio can be very close to 1, making convergence extremely slow for highly curved functions.

Intuition: Since f is convex, the chord lies above the curve, so its x -intercept is always to the right of x^* — the right bracket never tightens.

Improved Variants

Definition — Illinois Algorithm — Dowell & Jarratt, 1971

If the same endpoint (say b) is retained for two consecutive iterations, replace its function value with half:

$$\tilde{f}(b) \leftarrow \frac{f(b)}{2}$$

This artificially reduces the weight given to the stagnant endpoint, forcing c to move further into the bracket. Achieves **superlinear convergence** (order ≈ 1.44) while maintaining the bracket guarantee.

Definition — Pegasus Method — Dowell & Jarratt, 1972

When the same endpoint b is retained, replace:

$$\tilde{f}(b) \leftarrow f(b) \cdot \frac{f(b_{\text{old}})}{f(b_{\text{old}}) + f(c)}$$

A more sophisticated weighting than Illinois, achieving convergence order ≈ 1.64 — very close to the Secant Method's golden-ratio order.

Definition — Anderson-Björck Method — 1973

Adapts the weight dynamically:

$$\tilde{f}(b) \leftarrow f(b) \cdot \max\left(\frac{1}{2}, 1 - \frac{f(c)}{f(b)}\right)$$

Also achieves order ≈ 1.64 and adapts based on current function values.

Method	Conv. Order	Bracket?	Extra Cost	Note
Regula Falsi (plain)	1 (stagnates)	Yes	None	May be very slow
Illinois	≈ 1.44	Yes	One test/step	Simple fix
Pegasus	≈ 1.64	Yes	One multiply	Best of variants
Anderson-Björck	≈ 1.64	Yes	One comparison	Adaptive
Brent's Method	up to ≈ 1.618	Yes	Moderate	Production gold standard

Engineering Applications

Regula Falsi and its variants are widely used because they guarantee convergence without requiring derivatives:

- **Pipe-flow:** Colebrook–White equation for friction factor (transcendental in f).
- **Thermodynamics:** Van der Waals equation of state for V .
- **Circuit analysis:** Diode equations with exponential nonlinearities.
- **Structural mechanics:** Buckling load equations.

Advantages

- Bracket always maintained (safe)
- Uses slope information (smarter than bisection)
- Faster than bisection in most cases
- Guaranteed convergence
- No derivative required

Disadvantages

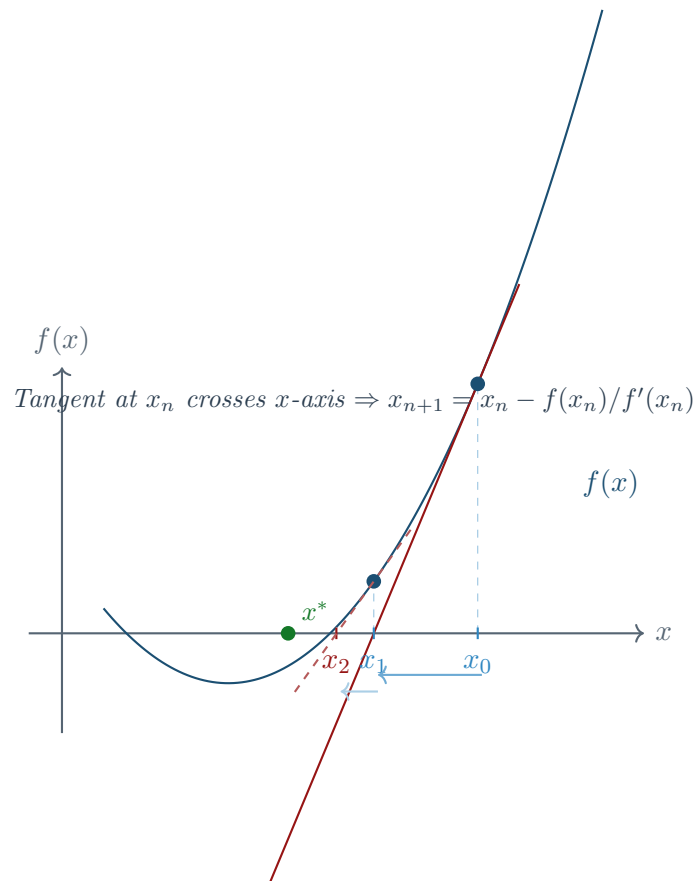
- One endpoint may stagnate (slow)
- Still only linear convergence
- Slower than Newton-Raphson / Secant
- Requires initial bracket
- Poor for nearly flat f near root

METHOD 03

Newton-Raphson Method

Historical Background

Isaac Newton (1669, *De analysi*) introduced a version of this method for polynomials. **Joseph Raphson** (1690) independently published a simplified, more general algebraic form. The modern calculus-based form using $f'(x)$ was developed by **Thomas Simpson** (1740). The method is now considered the cornerstone of computational root finding and is the basis of numerical solvers in MATLAB, Python's `scipy.optimize`, and virtually every scientific computing library.



Derivation

Derivation — From Taylor Series Expansion

Expand $f(x)$ in a Taylor series about the current estimate x_n :

$$f(x) = f(x_n) + (x - x_n)f'(x_n) + \frac{(x - x_n)^2}{2!}f''(x_n) + \dots$$

Truncate after the **linear term** (linear approximation):

$$f(x) \approx f(x_n) + (x - x_n)f'(x_n)$$

Set $f(x) = 0$ and solve for x :

$$0 = f(x_n) + (x_{n+1} - x_n)f'(x_n)$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Definition — Geometric Interpretation

At each step, draw the **tangent line** to the curve $y = f(x)$ at the point $(x_n, f(x_n))$. The tangent line has equation:

$$y - f(x_n) = f'(x_n)(x - x_n)$$

Setting $y = 0$ gives x_{n+1} . The method thus “slides down the slope” toward the root. This is why the method fails when $f'(x_n) = 0$ (horizontal tangent).

Convergence Analysis

Theorem — Quadratic Convergence

Assume f has a **simple root** x^* (i.e., $f(x^*) = 0$ and $f'(x^*) \neq 0$), and that f'' is continuous near x^* . Then Newton-Raphson converges **quadratically**:

$$|e_{n+1}| \approx \frac{|f''(x^*)|}{2|f'(x^*)|}|e_n|^2 \quad \text{where } e_n = x_n - x^*$$

The constant $C = \frac{|f''(x^*)|}{2|f'(x^*)|}$ is called the **asymptotic error constant**.

Practical consequence: If $|e_n| = 10^{-3}$, then $|e_{n+1}| \approx C \times 10^{-6}$ — the number of correct decimal digits approximately *doubles* at each step.

Convergence Theorems

Theorem — Local Convergence Theorem

Suppose $f : [a, b] \rightarrow \mathbb{R}$ is twice continuously differentiable, $f(x^*) = 0$, and $f'(x^*) \neq 0$. Then there exists $\delta > 0$ such that for any x_0 with $|x_0 - x^*| < \delta$, Newton-Raphson converges quadratically to x^* .

Proof sketch: Since f' is continuous and $f'(x^*) \neq 0$, there exists $\delta > 0$ with $|f'(x)| \geq m > 0$ on $[x^* - \delta, x^* + \delta]$ and $|f''(x)| \leq M$. Then:

$$|e_{n+1}| = \frac{|f''(\xi_n)|}{2|f'(x_n)|} |e_n|^2 \leq \frac{M}{2m} |e_n|^2$$

If $|e_0| < 2m/M$, then $|e_n| < \delta$ for all n by induction. □

Theorem — Fourier–Kantorovich Sufficient Condition

If f, f', f'' are continuous on $[a, b]$, $f(a) \cdot f(b) < 0$, and for all $x \in [a, b]$:

$$\frac{f(x) f''(x)}{[f'(x)]^2} < 1$$

Then Newton-Raphson converges monotonically to x^* , provided x_0 is chosen as the endpoint satisfying $f(x_0) \cdot f''(x_0) > 0$ (i.e. f and f'' have the same sign).

Failure Cases

When Newton-Raphson Fails or Diverges:

1. $f'(x_n) = 0$: Division by zero — method is undefined. Occurs at stationary points.
2. **Inflection point near root**: Tangent lines “shoot off” to infinity.
3. **Cycling**: For some functions and starting points, the iteration oscillates between two values.
4. **Divergence**: If x_0 is far from x^* or f is highly nonlinear, the method may diverge.
5. **Wrong root**: May converge to a different root than intended.

Newton-Raphson for Systems of Equations

Newton-Raphson generalises to systems $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ where $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - J(\mathbf{x}_n)^{-1} \mathbf{F}(\mathbf{x}_n)$$

where $J(\mathbf{x}_n)$ is the **Jacobian matrix** with entries $J_{ij} = \partial F_i / \partial x_j$. In practice, one solves the linear system $J(\mathbf{x}_n) \boldsymbol{\delta} = -\mathbf{F}(\mathbf{x}_n)$ and sets $\mathbf{x}_{n+1} = \mathbf{x}_n + \boldsymbol{\delta}$.

Halley's Method: Cubic Convergence

Definition — Halley's Method — Edmond Halley, 1694

Uses a second-order Taylor approximation (osculating parabola) instead of a tangent line:

$$x_{n+1} = x_n - \frac{2f(x_n)f'(x_n)}{2[f'(x_n)]^2 - f(x_n)f''(x_n)}$$

Convergence order is $p = 3$: $|e_{n+1}| \approx \frac{|f'''(x^*)|}{3|f'(x^*)|}|e_n|^3$. Correct digits triple each step.

However, f'' is also required, increasing computational cost.

Basin of Attraction

For complex-valued functions, the set of starting points converging to a particular root under Newton iteration forms the **basin of attraction** for that root. For polynomials, these basins are **fractals** — the famous *Newton fractals* — with boundaries intimately related to Julia sets in complex dynamics. This demonstrates that even a simple algorithm can exhibit extraordinary sensitivity to initial conditions.

Modified Newton-Raphson for Multiple Roots

When x^* is a root of **multiplicity** m (i.e., $f(x^*) = f'(x^*) = \dots = f^{(m-1)}(x^*) = 0$), standard Newton-Raphson degenerates to *linear* convergence. The fix:

Definition — Modified Formula for Multiple Roots

$$x_{n+1} = x_n - m \cdot \frac{f(x_n)}{f'(x_n)}$$

or equivalently, define $u(x) = \frac{f(x)}{f'(x)}$ and apply Newton to $u(x) = 0$:

$$x_{n+1} = x_n - \frac{u(x_n)}{u'(x_n)} = x_n - \frac{f(x_n)f'(x_n)}{[f'(x_n)]^2 - f(x_n)f''(x_n)}$$

This **restores quadratic convergence** for multiple roots.

Advantages

- Quadratic convergence (very fast)
- Correct digits double each step
- Foundation of many advanced solvers
- Easily extended to systems of equations
- Works in complex plane

Disadvantages

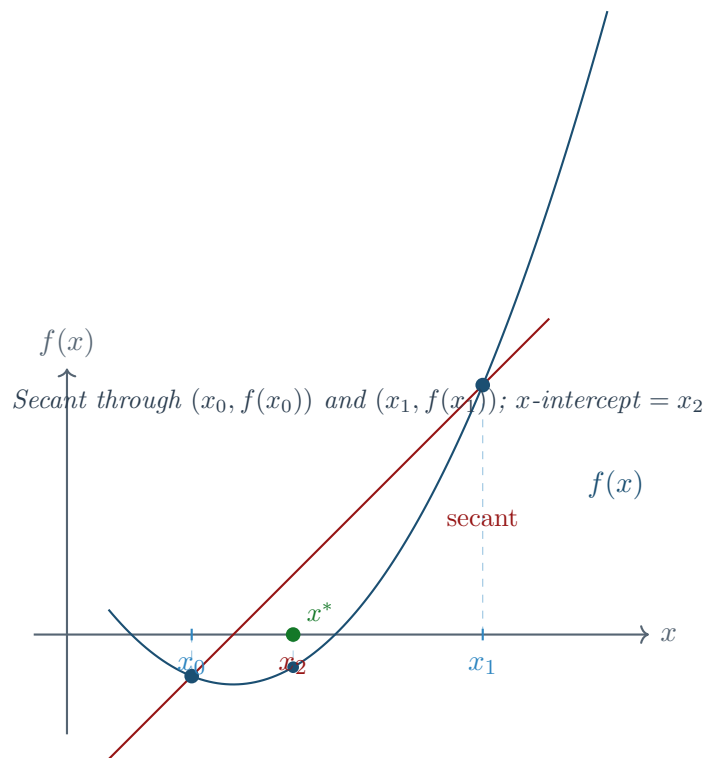
- Requires $f'(x)$ (may be hard to compute)
- Only local convergence (sensitive to x_0)
- Fails if $f'(x_n) = 0$
- May converge to wrong root
- Slow for multiple roots (unless modified)

METHOD 04

Secant Method

Historical Background

The Secant Method is closely related to Regula Falsi and has roots going back to ancient times, but its modern derivative-free formulation is attributed to **18th-century numerical analysts**. The method gained prominence as a practical alternative to Newton-Raphson when derivatives are difficult to compute symbolically. It is particularly important in **quasi-Newton optimization methods** (BFGS, SR1) and in the solution of nonlinear systems. The remarkable result that its convergence order is the *golden ratio* $\varphi = \frac{1+\sqrt{5}}{2}$ was established through careful error analysis.



Derivation

Derivation — Finite Difference Approximation of Derivative

Newton-Raphson requires $f'(x_n)$. In the Secant Method, we **approximate** the derivative using a finite difference between the two most recent iterates:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

Substituting into the Newton-Raphson formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{\frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}}$$

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Geometrically, this is the x -intercept of the **secant line** through $(x_{n-1}, f(x_{n-1}))$ and $(x_n, f(x_n))$.

Secant vs. Regula Falsi

Feature	Regula Falsi	Secant Method
Bracket required	Yes	No
Points used	a and b (bracket ends)	x_{n-1} and x_n (last two)
Convergence	Linear (may stagnate)	Superlinear
Sign change enforced	Yes	No
Guaranteed convergence	Yes	No

Convergence Analysis

Theorem — Superlinear Convergence — Golden Ratio Order

For a simple root x^* with $f'(x^*) \neq 0$, the Secant Method converges with order $p = \varphi = (1 + \sqrt{5})/2 \approx 1.618$.

Derivation — Proof: Error Recurrence and the Fibonacci Connection

Let $e_n = x_n - x^*$. Using the divided difference $f[x_{n-1}, x_n] = (f(x_n) - f(x_{n-1})) / (x_n - x_{n-1})$ and expanding around x^* :

$$f(x_n) \approx f'(x^*)e_n + \frac{1}{2}f''(x^*)e_n^2, \quad f[x_{n-1}, x_n] \approx f'(x^*) + \frac{f''(x^*)}{2}(e_n + e_{n-1})$$

After simplification, the error satisfies:

$$e_{n+1} \approx -\frac{f''(x^*)}{2f'(x^*)} e_n e_{n-1} = -C e_n e_{n-1}$$

Assuming $|e_n| \approx A\lambda^{p^n}$, comparing powers gives $p^{n+1} = p^n + p^{n-1}$, i.e. $p^2 = p + 1$. This is exactly the **Fibonacci recurrence characteristic equation**, whose positive root is:

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618 \quad (\text{the Golden Ratio } \varphi)$$

Deep connection: The Secant error satisfies the same recurrence as Fibonacci numbers. This is why the convergence order equals φ . $\square$

Numerical Instability Warning: When $x_n \approx x_{n-1}$ (iterates are close), computing $f(x_n) - f(x_{n-1})$ causes **catastrophic cancellation**. If both values agree in the first 12 of 15 digits, the difference has only 3 correct digits — the Secant step becomes wildly inaccurate. **Remedy:** switch to Bisection when $|x_n - x_{n-1}|$ becomes very small. This is exactly what Brent's Method does.

Computational Efficiency

Function Evaluation Comparison:

Newton-Raphson needs 2 evaluations per step (f and f'), achieving quadratic convergence (order 2). The Secant Method needs only 1 evaluation per step (reusing $f(x_{n-1})$), achieving order 1.618.

In terms of evaluations per order gained, Secant is **more efficient** than Newton-Raphson:

$$\text{Newton: } p = 2 \text{ per 2 evals} \quad \leftrightarrow \quad \text{Secant: } p \approx 1.618 \text{ per 1 eval}$$

To get 10 digits with Newton: ≈ 7 evaluations. To get 10 digits with Secant: ≈ 9 evaluations. The difference is small, but Secant avoids symbolic differentiation entirely.

Extensions

Definition — Muller's Method — 1956

While Secant fits a line through *two* points, Muller's Method fits a **quadratic** through three consecutive iterates $(x_{n-2}, f(x_{n-2}))$, $(x_{n-1}, f(x_{n-1}))$, $(x_n, f(x_n))$ and takes the nearest root of this quadratic as x_{n+1} . Convergence order $p \approx 1.839$ (root of $p^3 = p^2 + p + 1$). Can find **complex roots** starting from real initial values.

Definition — Inverse Quadratic Interpolation (IQI)

Treats x as a function of f and fits a quadratic through $(f(x_{n-2}), x_{n-2})$, $(f(x_{n-1}), x_{n-1})$, $(f(x_n), x_n)$. Evaluating at $f = 0$ gives the next iterate. Order ≈ 1.839 , numerically stable (no complex arithmetic). Used as the primary step in Brent's Method.

Theorem — Brent's Method — The Practical Gold Standard

Richard Brent, 1973. A hybrid algorithm combining:

1. **Inverse Quadratic Interpolation** (fast, order ≈ 1.839) — used when it makes progress.
2. **Secant Method** (fast, order 1.618) — used as fallback.
3. **Bisection** (slow but guaranteed) — used when faster methods fail.

Maintains a bracket (guaranteed convergence) while achieving superlinear speed in smooth regions. Implemented as `scipy.optimize.brentq`, MATLAB's `fzero`, and in GNU Scientific Library.

Advantages

- No derivative required
- Superlinear convergence (≈ 1.618)
- Only 1 function evaluation per step
- Higher efficiency index than Newton
- Simple formula, easy to implement
- Muller extension finds complex roots

Disadvantages

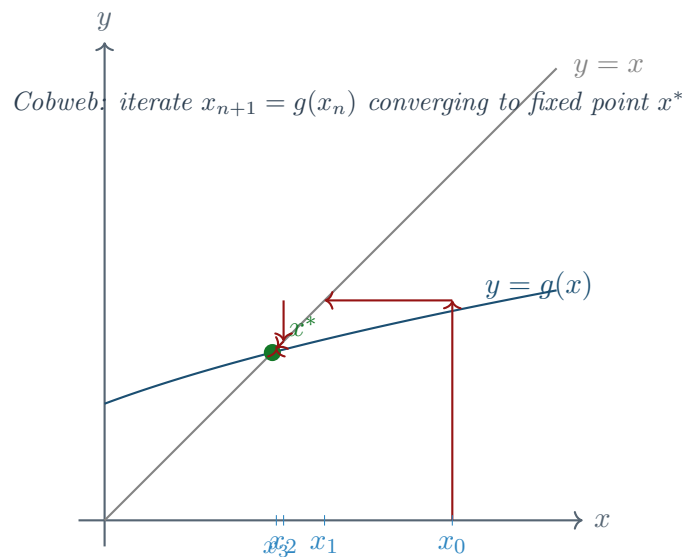
- No bracket maintained (may diverge)
- Needs two starting values x_0, x_1
- Catastrophic cancellation near convergence
- May converge to wrong root
- Slower than Newton per iteration
- Fails if $f(x_n) = f(x_{n-1})$

METHOD 05

Fixed Point Iteration

Historical Background

Fixed point theory is a cornerstone of modern mathematics. The **Banach Fixed Point Theorem** (Stefan Banach, 1922) provided the rigorous foundation for the convergence of iterative methods in complete metric spaces. L.E.J. Brouwer's topological fixed point theorem (1911) established existence in higher dimensions. In numerical analysis, fixed point iteration underpins many algorithms — including Newton-Raphson (as a special case), the Gauss-Seidel method for linear systems, and iterative solvers in computational physics. The concept of a *contraction mapping* is fundamental across pure and applied mathematics.



Mathematical Foundation

Definition — Fixed Point

A point x^* is a **fixed point** of a function g if:

$$g(x^*) = x^*$$

To find a root of $f(x) = 0$, we rewrite the equation as $x = g(x)$ for some function g . Any fixed point of g is then a root of f .

Examples of rearrangements for $f(x) = x^3 - 2x - 5 = 0$:

$$g_1(x) = \sqrt[3]{2x + 5} \quad (\text{converges}) \quad g_2(x) = \frac{x^3 - 5}{2} \quad (\text{diverges})$$

The key is choosing g so that $|g'(x^*)| < 1$.

Theorem — Banach Fixed Point Theorem (Contraction Mapping Theorem)

Let g be a **contraction mapping** on a closed interval $[a, b]$, meaning:

1. $g(x) \in [a, b]$ for all $x \in [a, b]$ (self-mapping)
2. $|g(x) - g(y)| \leq L|x - y|$ for all $x, y \in [a, b]$, where $0 \leq L < 1$ (Lipschitz condition)

Then:

- g has a **unique fixed point** x^* in $[a, b]$.
- The iteration $x_{n+1} = g(x_n)$ **converges** to x^* for any $x_0 \in [a, b]$.
- Error bound: $|x_n - x^*| \leq \frac{L^n}{1 - L} |x_1 - x_0|$

Derivation — Convergence Condition via Derivative

By the Mean Value Theorem, if g is differentiable on $[a, b]$:

$$g(x_n) - g(x^*) = g'(\xi_n)(x_n - x^*) \quad \text{for some } \xi_n \text{ between } x_n \text{ and } x^*$$

Therefore:

$$e_{n+1} = x_{n+1} - x^* = g(x_n) - g(x^*) = g'(\xi_n) e_n$$

As $x_n \rightarrow x^*$, we get $\xi_n \rightarrow x^*$, so asymptotically:

$$|e_{n+1}| \approx |g'(x^*)| |e_n|$$

The convergence rate is $|g'(x^*)|$, and convergence requires $|g'(x^*)| < 1$.

Choosing the Iteration Function $g(x)$

The choice of $g(x)$ is **critical**. Different rearrangements of the same equation can converge rapidly, converge slowly, or diverge completely.

Design principle: Given $f(x) = 0$, rearrange to $x = g(x)$ such that $|g'(x^*)| < 1$. The smaller $|g'(x^*)|$, the faster the convergence.

Connection to other methods: Newton-Raphson is a *special case* of Fixed Point Iteration with:

$$g(x) = x - \frac{f(x)}{f'(x)} \Rightarrow g'(x) = \frac{f(x)f''(x)}{[f'(x)]^2} \Rightarrow g'(x^*) = 0$$

Since $g'(x^*) = 0$, Newton-Raphson has **order 2** (quadratic convergence).

Order of Convergence

Theorem — General Convergence Order

If $g'(x^*) = g''(x^*) = \dots = g^{(p-1)}(x^*) = 0$ but $g^{(p)}(x^*) \neq 0$, then the iteration converges with order p :

$$e_{n+1} \approx \frac{g^{(p)}(x^*)}{p!} e_n^p$$

$g'(x^*)$	Order	Convergence Type
$0 < g'(x^*) < 1$	1	Linear
$g'(x^*) = 0, g''(x^*) \neq 0$	2	Quadratic
$g'(x^*) = g''(x^*) = 0, g'''(x^*) \neq 0$	3	Cubic

The Cobweb Diagram: Geometric Interpretation

Definition — Cobweb Diagram

A graphical tool for visualising fixed point iterations:

1. Plot $y = g(x)$ and $y = x$ (the diagonal).
2. Start at $(x_0, 0)$; draw vertically to the curve: reach (x_0, x_1) .
3. Draw horizontally to the diagonal: reach (x_1, x_1) .
4. Repeat: vertical to curve, horizontal to diagonal.

Converges (inward spiral): $|g'(x^*)| < 1$. If $0 < g'(x^*) < 1$: monotone staircase. If $-1 < g'(x^*) < 0$: oscillating spiral inward.

Diverges (outward spiral): $|g'(x^*)| > 1$ — cobweb spirals away from fixed point.

Newton-Raphson as a Fixed Point Iteration

Newton-Raphson is a special case with $g(x) = x - f(x)/f'(x)$. Computing g' :

$$g'(x) = 1 - \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}$$

At the root x^* where $f(x^*) = 0$:

$$g'(x^*) = \frac{0 \cdot f''(x^*)}{[f'(x^*)]^2} = 0$$

Since $g'(x^*) = 0$, Newton-Raphson achieves the best possible convergence factor — this is the algebraic explanation for its quadratic convergence.

Connections to Other Areas of Mathematics

Picard's Iteration for ODEs: Picard's theorem (existence & uniqueness of $y' = f(x, y)$) is proved via the fixed point iteration:

$$y_{n+1}(x) = y_0 + \int_{x_0}^x f(t, y_n(t)) dt$$

This is a fixed point iteration on the Banach space $C[x_0, x_0 + h]$, converging by the Banach theorem.

Power Iteration for Eigenvalues: To find the dominant eigenvalue of A , iterate $\mathbf{x}_{n+1} = A\mathbf{x}_n / \|A\mathbf{x}_n\|$. This is a fixed point iteration on the unit sphere in $\mathbb{R}^n$, converging to the eigenvector of λ_1 .

Gauss-Seidel as Fixed Point: Solving $A\mathbf{x} = \mathbf{b}$ by Gauss-Seidel gives the linear fixed point iteration $\mathbf{x}_{n+1} = G\mathbf{x}_n + \mathbf{c}$ with $G = -(D + L)^{-1}U$. Converges when the spectral radius $\rho(G) < 1$ — a direct application of the Banach contraction condition.

Steffensen's Method

A powerful acceleration technique that converts a linearly convergent fixed point iteration into a **quadratically convergent** one:

Definition — Steffensen's Method

Given x_n , compute two auxiliary values:

$$p = g(x_n), \quad q = g(p)$$

Then apply the **Aitken Δ^2 acceleration**:

$$x_{n+1} = x_n - \frac{(p - x_n)^2}{q - 2p + x_n}$$

This achieves quadratic convergence without requiring $f'(x)$, making it a powerful derivative-free alternative to Newton-Raphson.

Advantages

- Conceptually simple
- No derivative needed
- Very flexible (many choices of g)
- Foundation of advanced iterative solvers
- Can be accelerated (Steffensen's)

Disadvantages

- Convergence depends on choice of g
- Wrong g leads to divergence
- Usually only linear convergence
- No general rule for finding best g
- Slower than Newton-Raphson

Comprehensive Comparison of All Methods

Method	Conv. Order	Bracket?	Derivative?	Start Values	Key Property
Bisection	1 (ratio 0.5)	Yes	No	1 bracket	Guaranteed; slow
Regula Falsi	1 (varies)	Yes	No	1 bracket	Uses slope info
Newton-Raphson	2 (quadratic)	No	Yes f'	1 point	Fastest; local
Secant	1.618 (golden)	No	No	2 points	Fast; no deriv.
Fixed Point	1 (if $ g' < 1$)	No	No	1 point	Flexible; g matters

Summary of Error Formulas

Method	Error Formula / Bound
Bisection	$ e_n \leq \frac{b_0 - a_0}{2^{n+1}}$ (a priori, guaranteed)
Regula Falsi	$ e_{n+1} \approx \left(1 - \frac{f'(x^*)}{f'(x^*) + K}\right) e_n $ (linear; K curvature-related)
Newton-Raphson	$ e_{n+1} \approx \frac{ f''(x^*) }{2 f'(x^*) } e_n ^2$ (quadratic)
Secant	$ e_{n+1} \approx C e_n ^{1.618}$ (golden-ratio superlinear)
Fixed Point	$ e_{n+1} \approx g'(x^*) e_n $ (linear); $ x_n - x^* \leq \frac{L^n}{1-L} x_1 - x_0 $

Convergence Speed Illustration

Starting with error $|e_0| = 10^{-1}$ (1 correct digit), number of correct digits after n steps:

Step	Bisection	Regula Falsi	Newton-Raphson	Secant
1	~1 digit	~1–2 digits	~2 digits	~2 digits
2	~1 digit	~2 digits	~4 digits	~3 digits
3	~1 digit	~2–3 digits	~8 digits	~4 digits
4	~1 digit	~3 digits	~15 digits	~7 digits
5	~2 digits	~3–4 digits	machine zero	~11 digits

Digits Gained Per Function Evaluation

Method	Order p	Evals/step	Efficiency Index $p^{1/q}$	Digits/eval
Bisection	1 (factor 0.5)	1	1.00	0.30
Regula Falsi	≈ 1	1	≈ 1.00	≈ 0.30
Fixed Point	≈ 1	1	≈ 1.00	≈ 0.30
Newton-Raphson	2	2	1.414	0.65
Secant	1.618	1	1.618	0.72
Halley's Method	3	3	1.442	0.72
Steffensen	2	2	1.414	0.65

The Secant Method's efficiency index ≈ 1.618 is **higher** than Newton-Raphson's ≈ 1.414 , confirming that Secant extracts more accuracy per function evaluation. This makes it the method of choice when f' is expensive to compute.

Theoretical Properties at a Glance

Property	Bisection	Reg. Falsi	Newton	Secant	Fixed Pt
Guaranteed convergence	Yes	Yes	No	No	No
A priori error bound	Yes	No	No	No	Yes (Banach)
Global convergence	Yes	Yes	No	No	No
Derivative required	No	No	Yes	No	No
Multiple starting pts	No	No	No	Yes	No
Can find complex roots	No	No	Yes	No	No
Order of convergence	1	1	2	1.618	≥ 1
Extends to n -dim	No	No	Yes (Jacobian)	Partial	Yes

Practical Guidelines for Method Selection

1. **Bisection:** Safety and guaranteed convergence paramount; reliable bracket available; need known iteration count.
2. **Newton-Raphson:** $f'(x)$ available analytically; good initial guess near root; high precision required; working with systems (use Jacobian).
3. **Secant Method:** Derivative expensive or impossible; two reasonable starting points available; black-box function.
4. **Regula Falsi (Illinois/Pegasus):** Guaranteed convergence required but bisection too slow; bracket available.
5. **Fixed Point Iteration:** Equation naturally suggests $x = g(x)$ with small $|g'|$; or use Steffensen for derivative-free quadratic convergence.
6. **Brent's Method:** Default choice in production code (`fzero`, `brentq`) — bracket required but superlinear speed.

Summary of Key Formulas

Method	Iteration Formula
Bisection	$c_n = \frac{a_n + b_n}{2}$
Regula Falsi	$c_n = \frac{a_n f(b_n) - b_n f(a_n)}{f(b_n) - f(a_n)}$
Newton-Raphson	$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$
Modified Newton	$x_{n+1} = x_n - \frac{f(x_n)f'(x_n)}{[f'(x_n)]^2 - f(x_n)f''(x_n)}$
Halley	$x_{n+1} = x_n - \frac{2f(x_n)f'(x_n)}{2[f'(x_n)]^2 - f(x_n)f''(x_n)}$
Secant	$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$
Fixed Point	$x_{n+1} = g(x_n)$
Steffensen	$x_{n+1} = x_n - \frac{(g(x_n) - x_n)^2}{g(g(x_n)) - 2g(x_n) + x_n}$

NUMERICAL ANALYSIS I

System of Linear Equations

$A\mathbf{x} = \mathbf{b}$ — Direct and Iterative Methods

Methods Covered:

- | | |
|-------------------------|----------------------------|
| 1. Gaussian Elimination | 3. Jacobi Iterative Method |
| 2. LU Decomposition | 4. Gauss-Seidel Method |

Introduction to Systems of Linear Equations

Definition — System of Linear Equations

A **system of n linear equations** in n unknowns $x_1, x_2, \dots, x_n$ is written in matrix form as:

$$A\mathbf{x} = \mathbf{b}$$

where $A \in \mathbb{R}^{n \times n}$ is the **coefficient matrix**, $\mathbf{x} \in \mathbb{R}^n$ is the **unknown vector**, and $\mathbf{b} \in \mathbb{R}^n$ is the **right-hand side**. A unique solution exists if and only if $\det(A) \neq 0$.

Two classes of methods:

- **Direct methods** (Gaussian Elimination, LU Decomposition): find the exact solution in a finite number of steps. Best for small to medium dense systems.
- **Iterative methods** (Jacobi, Gauss-Seidel): start from an initial guess and converge to the solution. Best for large sparse systems.

METHOD 06

Gaussian Elimination

Historical Background

Carl Friedrich Gauss (1777–1855) systematized this method, though variants appear in ancient Chinese mathematics (*The Nine Chapters on the Mathematical Art*, circa 200 BCE). Gauss used it extensively in his least-squares work for astronomical calculations. The method is the foundation of virtually all direct linear solvers and underpins modern numerical libraries such as LAPACK and MATLAB's backslash operator.

Definition — Gaussian Elimination Algorithm

Given the augmented matrix $[A | \mathbf{b}]$, proceed in two phases:

Phase 1 — Forward Elimination: For each pivot row $k = 1, \dots, n - 1$:

$$R_i \leftarrow R_i - \frac{a_{ik}}{a_{kk}} R_k \quad \text{for } i = k + 1, \dots, n$$

This reduces A to upper triangular form U .

Phase 2 — Back Substitution:

$$x_n = \frac{b_n}{u_{nn}}, \quad x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^n u_{ij} x_j \right), \quad i = n - 1, \dots, 1$$

Theorem — Computational Complexity

Gaussian Elimination requires $\frac{2n^3}{3} + O(n^2)$ floating-point operations. **Partial pivoting** (swapping rows so the largest entry in each column becomes the pivot) is used in practice to avoid division by near-zero values and improve numerical stability.

Worked Example

Problem: Solve the system $A\mathbf{x} = \mathbf{b}$:

$$\begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ -11 \\ -3 \end{pmatrix}$$

Augmented matrix:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Forward elimination:

$$R_2 \leftarrow R_2 + \frac{3}{2}R_1: \left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

$$R_3 \leftarrow R_3 + R_1: \left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 2 & 1 & 5 \end{array} \right]$$

$$R_3 \leftarrow R_3 - 4R_2: \left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 0 & -1 & 1 \end{array} \right]$$

Back substitution:

$$x_3 = \frac{1}{-1} = -1, \quad x_2 = \frac{1 - \frac{1}{2}(-1)}{\frac{1}{2}} = 3, \quad x_1 = \frac{8 - (1)(3) - (-1)(-1)}{2} = 2$$

$$\boxed{x_1 = 2, \quad x_2 = 3, \quad x_3 = -1}$$

METHOD 07

LU Decomposition

Historical Background

LU decomposition was formalized by **Alan Turing** (1948) and independently developed by **Tadeusz Banachiewicz** (Crout's method, 1938) and **Prescott Doolittle** (Doolittle's method, 1878). It is the standard factorization used in LAPACK, MATLAB, and NumPy's `linalg.solve`. Its key advantage over raw Gaussian Elimination is that once $A = LU$ is computed, *multiple right-hand sides* can be solved cheaply.

Definition — LU Decomposition — Doolittle's Method

Factor $A = LU$ where:

$$L = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & 1 \end{pmatrix}, \quad U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix}$$

Entries are computed column by column:

$$u_{kj} = a_{kj} - \sum_{s=1}^{k-1} l_{ks} u_{sj}, \quad l_{ik} = \frac{1}{u_{kk}} \left(a_{ik} - \sum_{s=1}^{k-1} l_{is} u_{sk} \right)$$

To solve $A\mathbf{x} = \mathbf{b}$, solve $L\mathbf{y} = \mathbf{b}$ (forward substitution), then $U\mathbf{x} = \mathbf{y}$ (back substitution).

Worked Example

Problem: Decompose $A = \begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix}$ into LU and solve $A\mathbf{x} = (8, -11, -3)^T$.

Step 1 — Compute U (row 1) and L (column 1):

$$u_{1j} = a_{1j} : \quad U \text{ row 1} = [2, 1, -1]$$

$$l_{21} = \frac{-3}{2} = -\frac{3}{2}, \quad l_{31} = \frac{-2}{2} = -1$$

Step 2 — Row 2 of U and column 2 of L :

$$u_{22} = -1 - \left(-\frac{3}{2}\right)(1) = \frac{1}{2}, \quad u_{23} = 2 - \left(-\frac{3}{2}\right)(-1) = \frac{1}{2}$$

$$l_{32} = \frac{1 - (-1)(1)}{\frac{1}{2}} = 4$$

Step 3: $u_{33} = 2 - (-1)(-1) - (4)(\frac{1}{2}) = -1$

$$L = \begin{pmatrix} 1 & 0 & 0 \\ -\frac{3}{2} & 1 & 0 \\ -1 & 4 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & -1 \\ 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & -1 \end{pmatrix}$$

Forward substitution $Ly = \mathbf{b}$: $y_1 = 8, y_2 = 1, y_3 = 1$. **Back substitution** $U\mathbf{x} = \mathbf{y}$:
 $x_1 = 2, x_2 = 3, x_3 = -1$.

METHOD 08

Jacobi Iterative Method

Historical Background

Carl Gustav Jacob Jacobi (1801–1851) developed this iterative scheme for solving diagonally dominant systems. It belongs to the class of *stationary iterative methods* and is the conceptual predecessor to more powerful solvers like Gauss-Seidel and SOR. The method is naturally parallelizable since all updated values are computed simultaneously from the previous iteration, making it relevant in modern GPU-based computing.

Definition — Jacobi Iteration Formula

Split $A = D + (L + U)$ where D is the diagonal. The iteration is:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n$$

All $x_i^{(k+1)}$ use only values from the **previous** iteration k . In matrix form: $\mathbf{x}^{(k+1)} = D^{-1}(\mathbf{b} - (L + U)\mathbf{x}^{(k)})$.

Theorem — Convergence Condition

The Jacobi method converges if A is **strictly diagonally dominant**:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad \text{for all } i$$

Equivalently, the spectral radius of the iteration matrix satisfies $\rho(D^{-1}(L + U)) < 1$.

Worked Example

Problem: Solve by Jacobi iteration, starting from $(x_1, x_2) = (0, 0)$:

$$5x_1 + x_2 = 17 \quad x_1 + 5x_2 = 13$$

(Exact solution: $x_1 = 3, x_2 = 2$)

Iteration formulas:

$$x_1^{(k+1)} = \frac{17 - x_2^{(k)}}{5}, \quad x_2^{(k+1)} = \frac{13 - x_1^{(k)}}{5}$$

k	$x_1^{(k)}$	$x_2^{(k)}$	Error $\ e\ $
0	0.0000	0.0000	—
1	3.4000	2.6000	1.0198
2	2.8800	1.9200	0.1697
3	3.0160	2.0240	0.0284
4	2.9952	1.9968	0.0057
5	3.0006	2.0010	0.0011

Converging to (3, 2) with linear convergence rate.

METHOD 09

Gauss-Seidel Method

Historical Background

The Gauss-Seidel method was described by **Carl Friedrich Gauss** in a letter to his student Gerling (1823) and later analysed by **Philipp Ludwig von Seidel** (1874). It improves upon Jacobi by using each newly computed x_i *immediately* in the same iteration. This typically doubles the convergence rate and halves the memory requirement. It is the basis of successive over-relaxation (SOR) and is widely used in finite-element solvers and iterative image reconstruction.

Definition — Gauss-Seidel Iteration Formula

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

Updated values $x_j^{(k+1)}$ (already computed in the *current* sweep) are used immediately. In matrix form: $\mathbf{x}^{(k+1)} = (D + L)^{-1}(\mathbf{b} - U\mathbf{x}^{(k)})$.

Derivation — Gauss-Seidel vs. Jacobi

Feature	Jacobi	Gauss-Seidel
Update rule	Uses $\mathbf{x}^{(k)}$ only	Uses latest available values
Convergence rate	Slower	$\approx 2\times$ faster
Parallelizable	Yes (all at once)	No (sequential)
Memory	Needs two vectors	One vector sufficient

Worked Example

Problem: Solve the same system using Gauss-Seidel, starting from $(0, 0)$:

$$5x_1 + x_2 = 17 \quad x_1 + 5x_2 = 13$$

Iteration formulas (using updated x_1 immediately for x_2):

$$x_1^{(k+1)} = \frac{17 - x_2^{(k)}}{5}, \quad x_2^{(k+1)} = \frac{13 - x_1^{(k+1)}}{5}$$

k	$x_1^{(k)}$	$x_2^{(k)}$	Error $\ e\ $
0	0.0000	0.0000	—
1	3.4000	1.9200	0.4123
2	3.0160	1.9968	0.0161
3	3.0006	1.9999	0.0006
4	3.0000	2.0000	$< 10^{-4}$

Converges in **4 iterations** vs. 5+ for Jacobi — noticeably faster on the same system.

Advantages

- Faster convergence than Jacobi
- Uses one vector (memory efficient)
- Foundation for SOR acceleration
- Guaranteed convergence for SPD matrices

Disadvantages

- Not easily parallelizable
- May diverge if not diagonally dominant
- Order of equations affects convergence
- Convergence not always guaranteed

Chapter 2

Interpolation

Finite Differences and Interpolation Formulas

Topics Covered:

1. Finite Differences (Forward, Backward, Central)
2. Newton's Forward Interpolation
3. Newton's Backward Interpolation
4. Gauss Forward Interpolation
5. Gauss Backward Interpolation
6. Stirling's Interpolation Formula
7. Bessel's Interpolation Formula
8. Lagrange's Interpolation Formula
9. Newton's Divided Difference Method

Introduction to Interpolation

Definition — Interpolation

Interpolation is the process of estimating the value of a function $f(x)$ at a point x that lies **between** known data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$. The idea is to construct a polynomial $P(x)$ that passes through all the given data points and use it to estimate values at intermediate points.

Why Interpolation? In practice, a function may be known only at discrete tabulated points (e.g. experimental data, astronomical tables). Interpolation lets us estimate values between tabulated entries without re-running the experiment.

Finite Differences

The foundation of all difference-based interpolation formulas is the concept of **finite differences**.

Definition — Forward Difference Operator Δ

For a function $y = f(x)$ tabulated at equally spaced points with step size h :

$$\Delta y_r = y_{r+1} - y_r \quad (\text{First forward difference})$$

Higher-order forward differences are defined recursively:

$$\Delta^2 y_r = \Delta y_{r+1} - \Delta y_r, \quad \Delta^3 y_r = \Delta^2 y_{r+1} - \Delta^2 y_r, \quad \dots$$

In general:

$$\Delta^n y_r = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} y_{r+k}$$

Definition — Backward Difference Operator ∇

$$\nabla y_r = y_r - y_{r-1} \quad (\text{First backward difference})$$

Higher orders:

$$\nabla^2 y_r = \nabla y_r - \nabla y_{r-1}, \quad \nabla^n y_r = \sum_{k=0}^n (-1)^k \binom{n}{k} y_{r-k}$$

Definition — Central Difference Operator δ

$$\delta y_r = y_{r+\frac{1}{2}} - y_{r-\frac{1}{2}}$$

In practice, central differences are expressed using the notation $\delta^2 y_r = y_{r+1} - 2y_r + y_{r-1}$.

Forward Difference Table

Given $n + 1$ data points, the forward difference table is constructed as:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$
x_0	y_0					
		Δy_0				
x_1	y_1		$\Delta^2 y_0$			
		Δy_1		$\Delta^3 y_0$		
x_2	y_2		$\Delta^2 y_1$		$\Delta^4 y_0$	
		Δy_2		$\Delta^3 y_1$		$\Delta^5 y_0$
x_3	y_3		$\Delta^2 y_2$		$\Delta^4 y_1$	
		Δy_3		$\Delta^3 y_2$		
x_4	y_4		$\Delta^2 y_3$			
		Δy_4				
x_5	y_5					

Backward Difference Table

x	y	∇y	$\nabla^2 y$	$\nabla^3 y$	$\nabla^4 y$	$\nabla^5 y$
x_0	y_0					
x_1	y_1	∇y_1				
x_2	y_2	∇y_2	$\nabla^2 y_2$			
x_3	y_3	∇y_3	$\nabla^2 y_3$	$\nabla^3 y_3$		
x_4	y_4	∇y_4	$\nabla^2 y_4$	$\nabla^3 y_4$	$\nabla^4 y_4$	
x_5	y_5	∇y_5	$\nabla^2 y_5$	$\nabla^3 y_5$	$\nabla^4 y_5$	$\nabla^5 y_5$

Newton's Forward Interpolation Formula

METHOD 01

Newton's Forward Interpolation

Historical Background

Newton's forward difference interpolation formula was developed by **Sir Isaac Newton** in 1676 in his work *Methodus Differentialis*. It is one of the earliest systematic methods for interpolation and forms the foundation of finite difference calculus. The formula is particularly suited for interpolating near the **beginning** of a data table.

Definition — Newton's Forward Interpolation Formula

Given $n + 1$ equally spaced data points with step size $h = x_{r+1} - x_r$, define:

$$p = \frac{x - x_0}{h}$$

Then the interpolating polynomial is:

$$y_p = y_0 + p \Delta y_0 + \frac{p(p-1)}{2!} \Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!} \Delta^3 y_0 + \cdots + \frac{p(p-1) \cdots (p-n+1)}{n!} \Delta^n y_0$$

where x_0 is the **initial value**, h is the **difference of arguments**, and $\Delta^k y_0$ are the forward differences of y_0 .

Derivation — Derivation via Newton's Forward Difference Formula

We seek a polynomial $P_n(x)$ of degree $\leq n$ passing through $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$. Substituting $x = x_0 + ph$ and using the forward difference operator:

$$y_0 = y_0$$

$$y_1 = y_0 + \Delta y_0$$

$$y_2 = y_0 + 2\Delta y_0 + \Delta^2 y_0$$

In general, by the binomial-type expansion using forward differences:

$$y_p = \sum_{k=0}^n \binom{p}{k} \Delta^k y_0 \quad \text{where} \quad \binom{p}{k} = \frac{p(p-1)(p-2)\cdots(p-k+1)}{k!}$$

When to use: Newton's Forward Formula is used when the value of x at which interpolation is required lies **near the beginning** of the data table (i.e. p is small and positive).

Theorem — Error Term

The truncation error in Newton's Forward Interpolation Formula after n terms is:

$$E_n = \frac{p(p-1)(p-2)\cdots(p-n)}{(n+1)!} h^{n+1} f^{(n+1)}(\xi)$$

for some $\xi \in (x_0, x_n)$.

Worked Example

Problem: Given the table below, find $f(1.5)$ using Newton's Forward Formula.

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$
1	1			
		3		
2	4		2	
		5		0
3	9		2	
		7		
4	16			

Solution: $x_0 = 1, h = 1, x = 1.5 \Rightarrow p = \frac{1.5 - 1}{1} = 0.5$

$$\begin{aligned}y_{0.5} &= 1 + (0.5)(3) + \frac{(0.5)(-0.5)}{2}(2) + \frac{(0.5)(-0.5)(-1.5)}{6}(0) \\&= 1 + 1.5 - 0.25 + 0 = \boxed{2.25}\end{aligned}$$

Newton's Backward Interpolation Formula

METHOD 02

Newton's Backward Interpolation

Historical Background

Newton's Backward Interpolation Formula is the companion to the forward formula, designed for interpolating near the **end** of a data table. It uses backward differences $\nabla^k y_n$ starting from the last tabulated point x_n . It is particularly useful in **extrapolation** just beyond the end of the table.

Definition — Newton's Backward Interpolation Formula

Define:

$$p = \frac{x - x_n}{h}$$

where x_n is the **last** tabulated value and $p \leq 0$ for interpolation. Then:

$$y_p = y_n + p \nabla y_n + \frac{p(p+1)}{2!} \nabla^2 y_n + \frac{p(p+1)(p+2)}{3!} \nabla^3 y_n + \cdots + \frac{p(p+1) \cdots (p+n-1)}{n!} \nabla^n y_n$$

where $\nabla^k y_n$ are the backward differences at x_n (the last diagonal of the backward difference table).

Key difference from Forward Formula:

- Forward: $p = (x - x_0)/h$, uses $\Delta^k y_0$ — best near the **start**.
- Backward: $p = (x - x_n)/h$, uses $\nabla^k y_n$ — best near the **end**.
- The sign pattern changes: forward uses $p(p-1)(p-2) \cdots$, backward uses $p(p+1)(p+2) \cdots$.

Theorem — Error Term

The truncation error after n terms is:

$$E_n = \frac{p(p+1)(p+2)\cdots(p+n)}{(n+1)!} h^{n+1} f^{(n+1)}(\xi)$$

for some $\xi \in (x_0, x_n)$.

Worked Example

Problem: Find $f(4.5)$ from the data below using Newton's Backward Interpolation Formula.

x	y	∇y	$\nabla^2 y$	$\nabla^3 y$	$\nabla^4 y$
1	1				
2	8	7			
3	27	19	12		
4	64	37	18	6	
5	125	61	24	6	0

Solution: $x_n = 5$, $h = 1$, $x = 4.5$:

$$p = \frac{x - x_n}{h} = \frac{4.5 - 5}{1} = -0.5$$

$$\begin{aligned} y_p &= 125 + (-0.5)(61) + \frac{(-0.5)(0.5)}{2}(24) + \frac{(-0.5)(0.5)(1.5)}{6}(6) + 0 \\ &= 125 - 30.5 - 3 - 0.375 = \boxed{91.125} \end{aligned}$$

(Exact: $4.5^3 = 91.125$ — no error since the data is a cubic polynomial.)

Central Differences — Gauss Forward Formula

METHOD 03

Gauss Forward Interpolation

Historical Background

Carl Friedrich Gauss (1777–1855) developed two central difference interpolation formulas — forward and backward — that are more accurate than Newton’s formulas when interpolating **near the middle** of a data table. Central difference formulas use differences from both sides of the interpolation point, making them more symmetric and typically more accurate.

Definition — Gauss Forward Interpolation Formula

Let x_0 be the central point and $p = (x - x_0)/h$, where $-1 < p < 1$ for best accuracy. The formula is:

$$y_p = y_0 + p \Delta y_0 + \frac{p(p-1)}{2!} \Delta^2 y_{-1} + \frac{p(p-1)(p+1)}{3!} \Delta^3 y_{-1} + \frac{p(p-1)(p+1)(p-2)}{4!} \Delta^4 y_{-2} + \dots$$

where the differences are taken along the **forward diagonal** from x_0 , using:

$$\Delta^{2k} y_{-k}, \quad \Delta^{2k+1} y_{-k} \quad (k = 0, 1, 2, \dots)$$

Derivation — Structure of the Gauss Forward Formula

The pattern of differences used in the Gauss Forward formula proceeds *forward* through the central difference table:

$$y_0 \rightarrow \Delta y_0 \rightarrow \Delta^2 y_{-1} \rightarrow \Delta^3 y_{-1} \rightarrow \Delta^4 y_{-2} \rightarrow \Delta^5 y_{-2} \rightarrow \dots$$

The even-order differences are taken **one step above** the central row; odd-order differences are taken **at or below** the central row.

Central Difference (Forward) Table

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$
x_{-2}	y_{-2}					
		Δy_{-2}				
x_{-1}	y_{-1}		$\Delta^2 y_{-2}$			
		Δy_{-1}		$\Delta^3 y_{-2}$		
x_0	y_0		$\Delta^2 y_{-1}$		$\Delta^4 y_{-2}$	
		Δy_0		$\Delta^3 y_{-1}$		$\Delta^5 y_{-2}$
x_1	y_1		$\Delta^2 y_0$		$\Delta^4 y_{-1}$	
		Δy_1		$\Delta^3 y_0$		
x_2	y_2		$\Delta^2 y_1$			

Theorem — Error Term

The truncation error in Gauss Forward Interpolation after n terms is:

$$E_n = \frac{p(p^2 - 1)(p^2 - 4) \cdots}{(n + 1)!} h^{n+1} f^{(n+1)}(\xi), \quad \xi \in (x_{-\lfloor n/2 \rfloor}, x_{\lceil n/2 \rceil})$$

Worked Example

Problem: From the following table of $\sin \theta$, find $\sin(35)$ using Gauss Forward Interpolation.

θ	$\sin \theta$	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$
10	0.1736				
		0.1684			
20	0.3420		-0.0104		
		0.1580		-0.0048	
30	0.5000		-0.0152		0.0004
		0.1428		-0.0044	
40	0.6428		-0.0196		
		0.1232			
50	0.7660				

Solution: Take $x_0 = 30$, $h = 10$, $x = 35$:

$$p = \frac{35 - 30}{10} = 0.5$$

Gauss Forward differences from $x_0 = 30$: $\Delta y_0 = 0.1428$, $\Delta^2 y_{-1} = -0.0152$, $\Delta^3 y_{-1} = -0.0044$, $\Delta^4 y_{-2} = 0.0004$.

$$\begin{aligned}y_{0.5} &= 0.5000 + (0.5)(0.1428) + \frac{(0.5)(-0.5)}{2}(-0.0152) + \frac{(0.5)(-0.5)(1.5)}{6}(-0.0044) \\&\quad + \frac{(0.5)(-0.5)(1.5)(-1.5)}{24}(0.0004) \\&= 0.5000 + 0.07140 + 0.00190 + 0.000275 + 0.000009 \\&\approx \boxed{0.5736}\end{aligned}$$

(Actual: $\sin 35 = 0.5736$)

Central Differences — Gauss Backward Formula

METHOD 04

Gauss Backward Interpolation

Historical Background

The Gauss Backward formula is the second of Gauss's two central difference formulas. While the forward formula uses differences proceeding forward from the central row, the backward formula uses differences proceeding **backward** — making it more accurate when the interpolation point is slightly **below** the central value.

Definition — Gauss Backward Interpolation Formula

With $p = (x - x_0)/h$ and x_0 as the central value:

$$y_p = y_0 + p \nabla y_0 + \frac{p(p+1)}{2!} \nabla^2 y_0 + \frac{p(p+1)(p-1)}{3!} \nabla^3 y_0 + \frac{p(p+1)(p-1)(p+2)}{4!} \nabla^4 y_0 + \dots$$

where the differences proceed **backward** through the central difference table:

$$\nabla y_0 \rightarrow \nabla^2 y_0 \rightarrow \nabla^3 y_{-1} \rightarrow \nabla^4 y_{-1} \rightarrow \dots$$

Derivation — Gauss Forward vs. Backward — Key Comparison

Feature	Gauss Forward	Gauss Backward
Starting point	y_0	y_0
First difference	Δy_0	∇y_0
Direction	Forward diagonal	Backward diagonal
Best used when	$p > 0$ (above centre)	$p < 0$ (below centre)
2nd term sign	$p(p-1)$	$p(p+1)$

Worked Example

Problem: Using the same $\sin \theta$ table, find $\sin(25)$ using Gauss Backward Interpolation.

Solution: Take $x_0 = 30$, $h = 10$, $x = 25$:

$$p = \frac{25 - 30}{10} = -0.5$$

Gauss Backward differences from $x_0 = 30$ (backward diagonal):

$$\nabla y_0 = 0.1580, \quad \nabla^2 y_0 = -0.0104, \quad \nabla^3 y_{-1} = -0.0048, \quad \nabla^4 y_{-1} = 0.0004$$

(where $\nabla y_0 = \Delta y_{-1}$, $\nabla^2 y_0 = \Delta^2 y_{-2}$, $\nabla^3 y_{-1} = \Delta^3 y_{-2}$, $\nabla^4 y_{-1} = \Delta^4 y_{-2}$.)

$$\begin{aligned} y_{-0.5} &= 0.5000 + (-0.5)(0.1580) + \frac{(-0.5)(0.5)}{2}(-0.0104) + \frac{(-0.5)(0.5)(-1.5)}{6}(-0.0048) \\ &\quad + \frac{(-0.5)(0.5)(-1.5)(1.5)}{24}(0.0004) \\ &= 0.5000 - 0.0790 + 0.0013 - 0.0003 + 0.000009 \\ &\approx \boxed{0.4220} \end{aligned}$$

(Actual: $\sin 25 = 0.4226$ — small discrepancy due to rounding in the 4-decimal input table.)

Central Difference Advantage: Both Gauss formulas are more accurate than Newton's forward/backward formulas when interpolating near the **middle** of a table, because they draw differences from both sides of the interpolation point, reducing the asymmetry of the approximation.

Comprehensive Comparison of Interpolation Formulas

Formula	Operator	Differences Used	Best For	p Range
Newton Forward	Δ	$\Delta^k y_0$	Near start of table	$0 < p < 1$
Newton Backward	∇	$\nabla^k y_n$	Near end of table	$-1 < p < 0$
Gauss Forward	Δ	Forward diagonal from y_0	Near middle, $p > 0$	$0 < p < 1$
Gauss Backward	∇	Backward diagonal from y_0	Near middle, $p < 0$	$-1 < p < 0$

Summary of All Formulas

Formula	Expression
Newton Forward	$y_p = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \dots$
Newton Backward	$y_p = y_n + p\nabla y_n + \frac{p(p+1)}{2!}\nabla^2 y_n + \frac{p(p+1)(p+2)}{3!}\nabla^3 y_n + \dots$
Gauss Forward	$y_p = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_{-1} + \frac{p(p-1)(p+1)}{3!}\Delta^3 y_{-1} + \dots$
Gauss Backward	$y_p = y_0 + p\nabla y_0 + \frac{p(p+1)}{2!}\nabla^2 y_0 + \frac{p(p+1)(p-1)}{3!}\nabla^3 y_0 + \dots$

Stirling's Interpolation Formula

Central Differences

Stirling's Interpolation Formula

Definition — Central Differences — Stirling's Formula

Stirling's formula is a **central difference** interpolation formula derived by taking the arithmetic mean of the Gauss Forward and Gauss Backward formulae. It is most accurate when the interpolation parameter $p = \frac{x - x_0}{h}$ is close to **0**, i.e. when x is near the centre of the table.

Stirling's Difference Table

The table is centred at x_0 , using values symmetrically on both sides:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$
x_{-2}	y_{-2}					
		Δy_{-2}				
x_{-1}	y_{-1}		$\Delta^2 y_{-2}$			
		Δy_{-1}		$\Delta^3 y_{-2}$		
x_0	y_0		$\Delta^2 y_{-1}$		$\Delta^4 y_{-2}$	
		Δy_0		$\Delta^3 y_{-1}$		$\Delta^5 y_{-2}$
x_1	y_1		$\Delta^2 y_0$		$\Delta^4 y_{-1}$	
		Δy_1		$\Delta^3 y_0$		
x_2	y_2		$\Delta^2 y_1$			

Derivation

Derivation — Stirling from Gauss Forward + Backward

Gauss Forward:

$$y_p = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{(p+1)p(p-1)}{3!}\Delta^3 y_0 + \frac{(p+1)p(p-1)(p-2)}{4!}\Delta^4 y_0 + \dots$$

Gauss Backward:

$$y_p = y_0 + p\nabla y_0 + \frac{p(p+1)}{2!}\nabla^2 y_0 + \frac{(p+1)p(p-1)}{3!}\nabla^3 y_0 + \frac{(p+2)(p+1)p(p-1)}{4!}\nabla^4 y_0 + \dots$$

Adding both and dividing by 2 gives **Stirling's formula**.

Theorem — Stirling's Interpolation Formula

$$y_p = y_0 + p \left[\frac{\Delta y_0 + \Delta y_{-1}}{2} \right] + \frac{p^2}{2!} \Delta^2 y_0 + \frac{p(p^2-1)}{3!} \left[\frac{\Delta^3 y_{-1} + \Delta^3 y_{-2}}{2} \right] \\ + \frac{p^2(p^2-1)}{4!} \Delta^4 y_0 + \frac{p(p^2-1)(p^2-4)}{5!} \left[\frac{\Delta^5 y_{-2} + \Delta^5 y_{-3}}{2} \right] + \dots$$

where $p = \frac{x - x_0}{h}$.

Notes:

1. **Odd-factorial** terms use the *mean* of two adjacent differences (from both sides of x_0).
2. **Even-factorial** terms use a single central difference.
3. Odd factorial coefficients involve p ; even factorial coefficients involve p^2 .

Solved Example

Stirling's Formula — Find $y(1.3)$

Given:

x	1.0	1.2	1.4	1.6	1.8
$y = e^x$	2.7183	3.3201	4.0552	4.9530	6.0496

$$h = 0.2, \text{ take } x_0 = 1.4, p = \frac{1.3 - 1.4}{0.2} = -0.5.$$

Difference Table:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$
1.0	2.7183				
		0.6018			
1.2	3.3201		0.1333		
		0.7351		0.0294	
1.4	4.0552		0.1627		0.0066
		0.8978		0.0360	
1.6	4.9530		0.1987		
		1.0966			
1.8	6.0496				

At $x_0 = 1.4$: $\Delta y_0 = 0.8978$, $\Delta y_{-1} = 0.7351$, $\Delta^2 y_{-1} = 0.1627$,
 $\Delta^3 y_{-1} = 0.0360$, $\Delta^3 y_{-2} = 0.0294$, $\Delta^4 y_{-2} = 0.0066$.

$$\begin{aligned}
 y_{1.3} &= 4.0552 + (-0.5) \left[\frac{0.8978 + 0.7351}{2} \right] + \frac{0.25}{2} (0.1627) \\
 &\quad + \frac{(-0.5)(0.25 - 1)}{6} \left[\frac{0.0360 + 0.0294}{2} \right] + \frac{0.25(0.25 - 1)}{24} (0.0066) \\
 &= 4.0552 - 0.4082 + 0.0203 + 0.0020 - 0.00005 \\
 &\approx \boxed{3.6693}
 \end{aligned}$$

(Exact: $e^{1.3} = 3.6693 \checkmark$)

Bessel's Interpolation Formula

Central Differences

Bessel's Interpolation Formula

Definition — When to Use Bessel's Formula

Bessel's formula is a central difference interpolation formula most accurate when $p \approx \frac{1}{2}$, i.e. when the interpolation point lies near the **midpoint** between two consecutive tabulated values.

Derivation

Derivation — Bessel from Gauss Forward

Starting from Gauss Forward and using the identities:

$$\Delta^3 y_{-1} = \Delta^2 y_0 - \Delta^2 y_{-1}, \quad \Delta^5 y_{-2} = \Delta^4 y_{-1} - \Delta^4 y_{-2}$$

and substituting into Gauss Forward, then rearranging, yields Bessel's formula.

Theorem — Bessel's Interpolation Formula

$$y_p = \frac{y_0 + y_1}{2} + \left(p - \frac{1}{2}\right) \Delta y_0 + \frac{p(p-1)}{2!} \cdot \frac{\Delta^2 y_{-1} + \Delta^2 y_0}{2} \\ + \frac{(p - \frac{1}{2}) p(p-1)}{3!} \Delta^3 y_{-1} + \frac{p(p-1)(p+1)(p-2)}{4!} \cdot \frac{\Delta^4 y_{-2} + \Delta^4 y_{-1}}{2} + \dots$$

where $p = \frac{x - x_0}{h}$ and y_1 is the tabulated value at $x_1 = x_0 + h$.

At $p = \frac{1}{2}$: all terms containing the factor $(p - \frac{1}{2})$ vanish, leaving only even-order difference terms. This makes the formula **maximally efficient** at the midpoint.

Solved Example

Bessel's Formula — Find $y(25)$ **Given:**

x	10	20	30	40
y	46	66	81	93

$$h = 10, \text{ take } x_0 = 20, \quad p = \frac{25 - 20}{10} = 0.5.$$

Difference Table:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$
10	46			
		20		
20	66		-5	
		15		2
30	81		-3	
		12		
40	93			

$$y_0 = 66, \quad y_1 = 81, \quad \Delta y_0 = 15, \quad \Delta^2 y_{-1} = -5, \quad \Delta^2 y_0 = -3, \quad \Delta^3 y_{-1} = 2.$$

$$\begin{aligned}
 y_{25} &= \frac{66 + 81}{2} + (0.5 - 0.5)(15) + \frac{(0.5)(0.5 - 1)}{2} \cdot \frac{-5 + (-3)}{2} + \frac{(0.5 - 0.5)(0.5)(0.5 - 1)}{6}(2) \\
 &= 73.5 + 0 + \frac{(-0.25)}{2}(-4) + 0 \\
 &= 73.5 + 0.5 = \boxed{74}
 \end{aligned}$$

Lagrange's Interpolation Formula

Unequally Spaced Data

Lagrange's Interpolation Formula

Definition — Lagrange Polynomials

Lagrange's method constructs an interpolating polynomial through $n + 1$ data points that may be **unequally spaced**. No difference table is needed. It is the standard method when the x -values are not uniformly distributed.

Formula

Theorem — Lagrange Interpolating Polynomial

Given $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$:

$$P(x) = \sum_{i=0}^n L_i(x) y_i = L_0(x) y_0 + L_1(x) y_1 + \dots + L_n(x) y_n$$

The **basis (cardinal) polynomials** are:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

Explicitly:

$$L_0(x) = \frac{(x - x_1)(x - x_2) \cdots (x - x_n)}{(x_0 - x_1)(x_0 - x_2) \cdots (x_0 - x_n)}$$

$$L_1(x) = \frac{(x - x_0)(x - x_2) \cdots (x - x_n)}{(x_1 - x_0)(x_1 - x_2) \cdots (x_1 - x_n)}$$

Key property: $L_i(x_j) = \delta_{ij}$ — each basis polynomial equals 1 at its own node and 0 at all others. This guarantees $P(x_i) = y_i$ for all i .

Solved Example

Lagrange's Formula — Find $y(1.5)$ **Given:** $y = x^3$

x	1	2	3	4
y	1	8	27	64

Find $y(1.5)$. **Compute basis polynomials at $x = 1.5$:**

$$L_0(1.5) = \frac{(1.5 - 2)(1.5 - 3)(1.5 - 4)}{(1 - 2)(1 - 3)(1 - 4)} = \frac{(-0.5)(-1.5)(-2.5)}{(-1)(-2)(-3)} = \frac{-1.875}{-6} = 0.3125$$

$$L_1(1.5) = \frac{(1.5 - 1)(1.5 - 3)(1.5 - 4)}{(2 - 1)(2 - 3)(2 - 4)} = \frac{(0.5)(-1.5)(-2.5)}{(1)(-1)(-2)} = \frac{1.875}{2} = 0.9375$$

$$L_2(1.5) = \frac{(1.5 - 1)(1.5 - 2)(1.5 - 4)}{(3 - 1)(3 - 2)(3 - 4)} = \frac{(0.5)(-0.5)(-2.5)}{(2)(1)(-1)} = \frac{0.625}{-2} = -0.3125$$

$$L_3(1.5) = \frac{(1.5 - 1)(1.5 - 2)(1.5 - 3)}{(4 - 1)(4 - 2)(4 - 3)} = \frac{(0.5)(-0.5)(-1.5)}{(3)(2)(1)} = \frac{0.375}{6} = 0.0625$$

Interpolated value:

$$P(1.5) = 1(0.3125) + 8(0.9375) + 27(-0.3125) + 64(0.0625) = 0.3125 + 7.5 - 8.4375 + 4 = \boxed{3.375}$$

(Exact: $(1.5)^3 = 3.375$ ✓)

Newton's Divided Difference Method

Unequally Spaced Data

Newton's Divided Difference Method

Historical Background

Lagrange's formula has the drawback that if a new data point is inserted, **all coefficients must be recomputed** from scratch. Newton's Divided Difference method avoids this by building the interpolating polynomial *incrementally* — adding a new point only requires one additional divided difference column.

Divided Differences

Definition — Divided Difference Notation

Let $y_0, y_1, \dots, y_n$ be values of $f(x)$ at distinct (not necessarily equally spaced) points $x_0, x_1, \dots, x_n$.

Zero-th order: $f[x_i] = y_i$

First order: $f[x_i, x_{i+1}] = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$

Second order: $f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}$

k -th order: $f[x_0, \dots, x_k] = \frac{f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]}{x_k - x_0}$

Theorem — Newton's Divided Difference Formula

$$P(x) = f[x_0] + (x - x_0) f[x_0, x_1] + (x - x_0)(x - x_1) f[x_0, x_1, x_2] \\ + (x - x_0)(x - x_1)(x - x_2) f[x_0, x_1, x_2, x_3] + \dots$$

Solved Example

Newton's Divided Difference — Find $f(2)$

Given:

x	0	1	3	4
$f(x)$	5	6	50	105

Divided Difference Table:

x	$f[x]$	$f[\cdot, \cdot]$	$f[\cdot, \cdot, \cdot]$	$f[\cdot, \cdot, \cdot, \cdot]$
0	5			
		$\frac{6-5}{1-0} = 1$		
1	6		$\frac{22-1}{3-0} = 7$	
		$\frac{50-6}{3-1} = 22$		$\frac{11-7}{4-0} = 1$
3	50		$\frac{55-22}{4-1} = 11$	
		$\frac{105-50}{4-3} = 55$		
4	105			

Newton's polynomial:

$$\begin{aligned}
 P(x) &= 5 + (x-0)(1) + (x-0)(x-1)(7) + (x-0)(x-1)(x-3)(1) \\
 &= 5 + x + 7x(x-1) + x(x-1)(x-3) \\
 &= x^3 + 3x^2 - 3x + 5
 \end{aligned}$$

Find $f(2)$:

$$P(2) = 5 + (2)(1) + (2)(1)(7) + (2)(1)(-1)(1) = 5 + 2 + 14 - 2 = \boxed{19}$$

Verify: $P(0) = 5$, $P(1) = 6$, $P(3) = 50$, $P(4) = 105$ ✓

Adding a new point (x_5, y_5) only requires computing one new divided difference column — all previous columns remain valid. This is the key advantage over Lagrange's method.

Comprehensive Comparison of All Interpolation Formulas

Formula	Spacing	Differences	Best For	p Range
Newton Forward	Equal	$\Delta^k y_0$	Near start of table	$0 < p < 1$
Newton Backward	Equal	$\nabla^k y_n$	Near end of table	$-1 < p < 0$
Gauss Forward	Equal	Forward diagonal	Middle, $p > 0$	$0 < p < 1$
Gauss Backward	Equal	Backward diagonal	Middle, $p < 0$	$-1 < p < 0$
Stirling's	Equal	Mean of fwd & bwd	Middle, $p \approx 0$	$-\frac{1}{2} < p < \frac{1}{2}$
Bessel's	Equal	Central (averaged)	Midpoint , $p \approx \frac{1}{2}$	$0 < p < 1$
Lagrange's	Unequal	None (direct)	Any point	Any
Newton Div. Diff.	Unequal	Divided differences	Any point	Any

Summary of All Formulas

Formula	Expression
Newton Forward	$y_p = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \dots$
Newton Backward	$y_p = y_n + p\nabla y_n + \frac{p(p+1)}{2!}\nabla^2 y_n + \frac{p(p+1)(p+2)}{3!}\nabla^3 y_n + \dots$
Gauss Forward	$y_p = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_{-1} + \frac{(p+1)p(p-1)}{3!}\Delta^3 y_{-1} + \dots$
Gauss Backward	$y_p = y_0 + p\nabla y_0 + \frac{p(p+1)}{2!}\nabla^2 y_0 + \frac{(p+1)p(p-1)}{3!}\nabla^3 y_0 + \dots$
Stirling's	$y_p = y_0 + p\left[\frac{\Delta y_0 + \Delta y_{-1}}{2}\right] + \frac{p^2}{2!}\Delta^2 y_{-1} + \frac{p(p^2-1)}{3!}\left[\frac{\Delta^3 y_{-1} + \Delta^3 y_{-2}}{2}\right] + \dots$
Bessel's	$y_p = \frac{y_0 + y_1}{2} + \left(p - \frac{1}{2}\right)\Delta y_0 + \frac{p(p-1)}{2!} \cdot \frac{\Delta^2 y_{-1} + \Delta^2 y_0}{2} + \dots$
Lagrange's	$P(x) = \sum_{i=0}^n L_i(x) y_i, \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$
Newton Div. Diff.	$P(x) = f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \dots$